From Scratch:
The Design and Implementation of a SAT Solver
From Scratch

Oliver Lie

August 13, 2026

Contents

Preface

Hello! Thank you for taking me seriously enough to pick up this book and begin reading it. Before we continue, I just want you to know that I am not an expert when it comes to things such as algorithms, data structures, general computer science, or even SAT solvers (what this book is about). Despite that, that's exactly why I wanted to write this. These things that I am not an expert in are my passion, and by doing this technical research, I hope to become a little less of "not an expert" in those fields.

The aim of this book isn't to improve or revolutionize SAT solving algorithms. Rather, it is to make the methods and reasoning more accessible to the lay undergraduate (I am open to the possibility that I myself may be the lay undergraduate, and you are simply smarter than me). And for the record, to cover myself in the future, if you are a student tasked with creating a SAT solver, DO NOT USE THIS BOOK OR ANY OF THE ONLINE CODE TO CHEAT!!! This book and the online code is meant to help you learn how these algorithms are implemented, not for you to copy.

Throughout this book we will build our own modern SAT solver from scratch, complete with all the bells and whistles. We will begin with the most basic algorithm imaginable, brute force, and gradually work our way through DPLL, CDCL, and beyond.

Throughout this journey we will encounter thought experiments, exercises, and a handful of experiments intended to solidify our understanding. Any external resources will be cited (hopefully correctly), and in keeping with the goal of accessibility:

- Questions that are explicitly asked will eventually be answered.
- Proofs will be explained in informal language before becoming formal.
- Code will be provided and linked through GitHub (or another hosting platform).
- Revisions to this book will be made whenever I discover mistakes or better explanations.

There is one more thing that deserves acknowledgement: some pieces of code were AI-assisted. To be completely transparent, "AI-assisted" means that instead of spending days repeatedly banging my head against a wall trying to understand a bug or some subtle algorithmic detail, I occasionally asked an AI to explain concepts or help debug an implementation. Every word of this book, however, is my own. AI did not generate any of the explanatory text you will read here, although it certainly helped produce some of the code that accompanies it.

There are practical reasons for this. SAT solvers are difficult to implement correctly, and introducing subtle bugs is remarkably easy. In fact, before deciding to write this book, I had already attempted to build a SAT solver once before. I never managed to get a working CDCL implementation.

That failure, however, taught me a tremendous amount. So in a sense, we are learning SAT solving together (isn't that comforting?). My previous mistakes have shown me many of the pitfalls, and my hope is that by documenting this journey I can help you avoid making the same ones. So without further ado, let's begin. By the time we reach the end, I hope I will have taught you more than I myself learned.

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, particle physics, solving graph problems, and much much more. [1]

In a more computer science-focused world, they have applications in NP-Completeness. Since (nearly) [2] every problem has a reduction to a Boolean SAT problem, creating one really really good SAT solver lets us solve all those problems in a "fast" manner. So they are important everywhere despite the vast majority of us never really thinking about their existence. One note on what "fast" means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn't necessarily "fast" as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we're all on the same page on things such as input and ideas. Reading through this introduction is important so that you understand the basic notation used throughout this book.

One last thing to note: we will write out pseudocode rather than the code for one specific programming language. However, although the idea behind pseudocode is that it's programming language agnostic (it doesn't carry artifacts of any real programming language (usually)), this idea is poorly executed when it comes to formatting. My algorithms professor will probably scream at me for this, but I'm making the executive decision that we will follow a general C-based syntax for readability purposes. That is, we will use braces "{}" for block scope, parentheses "()" for grouping, "=" for assignment, "==" for equality, and "<=", ">=", etc. for their respective uses. Everything that can be inferred visually will be used. Another thing to note is that we will pretend our arrays can dynamically resize, so we won't pre-allocate space, [3] and we will also assume 1-based indexing for an easier map from variables to indices, as you will see

[1] `https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf`

[2] Before anyone comes after me, the Halting Problem doesn't have a reduction to a boolean satisfiability problem, so take that.

[3] If you are implementing this in real code, this tiny detail of no pre-allocation of memory will come back to bite you later, so if you are encountering segfaults, this *should* be one of the first things you should check!

later.

1.1 What Do SAT Solvers Even Solve?

"What do SAT solvers even solve," I hear you say? SAT solvers solve *boolean satisfiability* problems. Another way to say that is they solve *propositional logic* problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \rightarrow Q$, an implication statement
- $(A \wedge B)$, an and statement
- $(A \vee B)$, an or statement
- $\neg A$, a negation/not statement

Of course these are very basic examples, and we haven't included every operator such as xor, nand, etc. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operate on Boolean algebra equations. [4]

However, SAT solvers don't operate on just any Boolean algebra equation. No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunctive Normal Form) equations are a "conjunction of disjunctions," i.e., clauses of ors, combined with ands.

Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are **not** CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form. [5]

[4] A Boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

[5] DNF (Disjunctive Normal Form) is a "disjunction of conjunctions," i.e., clauses of ands combined by ors. Solving these is trivially easy, as we just need to find one clause where no element evaluates to false.

1.2 Our Environment

Before we begin, we should talk about the environment that this book uses.

- Hardware: an Apple M1 MacBook Air, running MacOS Sequoia 15.7.8 with 8GB RAM and 512GB of storage space.
- Programming Language: C++20 compiled by Apple clang version 17.0.0 (clang-1700.6.4.2) targetted for arm64-apple-darwin24.6.0
- IDE: Visual Studio Code version 1.132.1
- Testing and Benchmarking: Testing and benchmarking is done by Test++, a C++ unit testing program I wrote (and is open source in case you want to try it out). [6] At the time of writing, it does not optimize the compiled code. It is also not technically a benchmarking program. The way tests are set up is that each category of test/testing-set is given its own Test++ test which is run to completion. The resulting time to complete that Test++ test is used as the benchmark time. Differences in benchmarking frameworks and the way tests are set up may affect the result, but our testing workflow is as follows:

  1. Create a Test++ testing function
  2. Read in the testing file(s) from the given file path/directory
  3. Create an instance of our SAT solver
  4. Solve the equation
  5. Check that the result is correct
  6. Repeat steps 3-5 until no more testing files need to be processed

1.3 CNF Encoding

Now that we know what our inputs look like, the next question (I hope you're asking questions) is "how do we encode that?" Well good thing you're reading this, because that's what we're here to talk about! In this section, we'll expand our base of boolean algebra, including some definitions, transformations, and finally good encoding.

Definitions We'll Use

Clause A clause is a collection of variables, in which each variable is separated by an or ($\vee$), and each clause is separated by an and ($\wedge$).

Variable A variable is an element of a clause, taking on a value of True, False, or Unassigned. Each variable can be negated ($\neg$) or not, affecting its evaluated value.

[6] Not a shameless plug because it's awesome.

Literal A literal is an evaluated variable, evaluating to one of True, False, or Unassigned. Since a literal is an evaluated variable, it cannot be negated. A literal whose corresponding variable is unassigned evaluates to Unassigned, regardless of whether or not the variable is negated.

A quick example of the difference of a literal value, using all three of our definitions:

$$(\neg A)$$

is a clause of one variable "A", which is negated. If the variable $A = False$, then its literal value is $True$, because

$$\neg False \equiv True.$$

In other words, A was assigned the value False, and then it evaluated to True. This will be very important in the future, so please take the time to understand it well enough.

As discussed earlier, a CNF equation only has three operators: and ($\wedge$), or ($\vee$), and not ($\neg$), but in propositional logic, there are many more operators, and their clauses are not guaranteed to be in the format we want. There are many algorithms to transform any propositional logic equation into CNF, but frankly, that's beyond the scope of this book (it's also beyond my scope). Instead, we will just look at how we transform each operator into an equivalent format using just our three operators.

Implication ($\rightarrow$)

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Xor ($\oplus$)

p	q	$\neg p$	$\neg q$	$p \oplus q$	$p \vee q$	$\neg p \vee \neg q$	$(p \vee q) \wedge (\neg p \vee \neg q)$
T	T	F	F	F	T	F	F
T	F	F	T	T	T	T	T
F	T	T	F	T	T	T	T
F	F	T	T	F	F	T	F

Biconditional ($\leftrightarrow$)

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(\neg p \vee q) \wedge (\neg q \vee p)$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	F	T	F	F
F	F	T	T	T	T

We won't look at negations of operators such as nand, nor, xnor, or any others. I'm not a boolean algebra expert when it comes to what operators exist. But if you were given an equation with these extra operators, you could easily apply the correct equivalency, then apply an algorithm that converts any boolean algebra equation into one of CNF form.[7]

DIMACS CNF Format

Now that we know what CNF looks like, we can now discuss how it's formatted for a SAT solver's consumption. Typically, CNF is formatted in *DIMACS CNF* form.

Some rules for DIMACS CNF form are:

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.
2. In some examples of CNF files, the definition of the last clause is not terminated by a final "0".
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with "c". [8]

For a quick example, here is a simple way to format a DIMACS CNF equation:

[7] I honestly may be talking out of my ass here. I will do more research on this subject at a later date.

[8] `https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html`

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment
1 2 -3 0
-1 4 0
```

In order to increase the compatibility of our SAT solver, we will be bending some rules.

We will firstly allow missing header lines. In the case where no header is passed in, we will instead infer the number of clauses and variables. We will also be very lenient in where comments can appear, but we will enforce that variables must be between 1 and N.

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with "c".
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with "0", and can extend multiple lines.
4. All variables are numbered 1 through N, where N is the number of variables. That is, if you say there are 10 variables, they must be labeled $1, 2, \ldots, 10$ and not $2, 3, \ldots, 11$ or any other convention.
5. (For compatibility) Upon encountering a "%" character outside of a comment, we will stop parsing immediately.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

```
4 2
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
c the first two numbers are still the number
c of variables and the number of clauses
```

```
p cnf 4 2 1 2 -3 0
-1 4 0
```

More information can be found here.[9]

1.4 Reading a CNF Equation Programmatically

Now that we know what our input looks like, we can finally discuss reading it programmatically. Firstly we must discuss how our SAT solver will be represented in code. At the moment, we just need to keep track of

- the number of variables
- the number of clauses
- each clause in our equation
- whether or not we've tried to solve the equation yet
- what our solution is

We also want some methods to parse the equation, construct a SAT solver, solve the equation, print out the solution if it is satisfiable, and whether or not there's a solution. Therefore, our object will look like so:

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      void parse(string equation)
7      int numVars
8      int numClauses
9      int[][] clauses
10     optional<bool>[] vars
```

Our pseudocode is as follows:

[9] https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html

```
SATSolver(string input)
{
    vars = no-value

    if (input is a file)
    {
        string content = readFile(input)
        parse(content)
    }
    else
    {
        parse(input)
    }
}
```

```
parse(equation)
{
    // The method of parsing if there's a header
    if (contains(equation, "p cnf"))
    {
        int[1..N] Arr

        for each (line in input)
        {
            if (startsWith(line, 'c'))
            {
                skip to next line
            }
            else
            {
                for each (token in line)
                {
                    append(Arr, token)
                }
            }
        }

        numVars = Arr[1]
        numClauses = Arr[2]

        idx = 2

        for (i = 1 up to numClauses)
        {
            int[1..N] clause

            while (Arr[idx] != 0)
            {
```

```
34                  append(clause, Arr[idx])
35                  idx = idx + 1
36              }
37
38              append(clauses, clause)
39          }
40      }
41
42      // Method of parsing when there's no header
43      else
44      {
45          numVars = 0
46          numClauses = 0
47
48          int[1..N] Arr
49
50          for each (line in input)
51          {
52              if (startsWith(line, 'c'))
53              {
54                  skip to next line
55              }
56              else
57              {
58                  for each (token in line)
59                  {
60                      numVars = max(numVars, token)
61
62                      if (token == 0)
63                      {
64                          numClauses = numClauses + 1
65                      }
66
67                      append(Arr, token)
68                  }
69
70                  idx = 0
71
72                  for (i = 1 up to numClauses)
73                  {
74                      int[1..N] clause
75
76                      while (Arr[idx] != 0)
77                      {
78                          append(clause, token)
79                          idx = idx + 1
80                      }
81
82                      append(clauses, clause)
83                  }
```

```
84              }
85          }
86      }
87  }
```

This code will be implemented slightly differently in the example code, as I am using C++. There is also a big bug, which is that the code assumes that because "p cnf" appears in the file, there is a header. This is not always true because it does not consider whether or not the header is on the same line as a comment, so this will break it:

```
c p cnf
1 2 -3 0
-1 4 0
```

Because it assumes that there is a header when in fact there is not. I will leave fixing this bug as an exercise to both the reader and the writer, however, I will not be implementing this fix because if you're purposely trying to break the constructor, you're not trying to solve a CNF equation anyways. In case you're wondering, the fix is to first find "p cnf" then scan backwards until you reach either the start of the file, or the end of the previous line. If you don't encounter a "c" then you know that this header is actually a header.

Chapter 2

Brute Force

Since the dawn of man, whenever encountered with a difficult problem, we have decided "eh, let's do it in the hardest way possible because I can't think of a better solution." The history of brute forcing things goes all the way back to our ancestors discovering fire, [1] all the way to our current solution to climate change. [2]

The idea of brute forcing something is really as simple as it gets. We have n number of literals, and therefore 2^n possible assignments, and we will check them all until we have either found a solution, or until we've run out of assignments to check. Because of how simple the idea is, it would just be better to show code and explain that instead of explaining then showing code.

```
1   solveCNF()
2   {
3       bool[1..numVars] Assignment
4
5       for (i = 0 up to 1 << numVars)
6       {
7           for (j = 0 up to numVars - 1)
8           {
9               Assignment[j] = i & (1 << j)
10          }
11
12          bool solved = true
13
14          for each (clause in clauses)
15          {
16              bool satisfied_clause = false
17
18              for (j = 1 up to clause.size)
19              {
20                  int variable = clause[j]
```

[1] This is probably not true.

[2] Which is doing nothing and hoping everything sorts itself out, which is really disappointing.

```
21                  int v_idx = abs(variable)
22
23                  if (variable > 0)
24                  {
25                      satisfied_clause = Assignment[v_idx]
26                        || satisfied_clause
27                  }
28                  else
29                  {
30                      satisfied_clause = !Assignment[v_idx]
31                        || satisfied_clause
32                  }
33              }
34
35              solved &= satisfied_clause
36          }
37
38          if (solved == true)
39          {
40              vars = Assignment
41              return true
42          }
43      }
44
45      return false
46  }
```

Unfortunately, the formatting across the page kind of sucks, but let's try to explain it anyways.

First, we create an array to hold each boolean value, and then next we start our loop from 0 all the way to "1 << numVars." If you are unfamiliar with what the left bit shift operator (<<) is, essentially, it is a fast way of computing some number times 2 times the number of places to shift. That is all the explanation I am going to give because that's not the point. But by computing "1 << numVars", we are computing $1 \rightarrow 2^n$, which happens to be the total number of possible variable combinations (from all False to all True).

Then in the first inner for loop, we put the current variable combination into the 'Assignment' array. The index variable 'i' holds the current variable combination, and we isolate the jth variable using "1 << j" and put it into the jth spot in the "Assignment" array.

Next, we evaluate the CNF equation against the current assignment. We first assume that the equation is solved (innocent until proven guilty), and then we evaluate each clause. We initially assume the clause is false because logically, if one literal is True, then OR-ing it with False makes the "clause satisfied" condition true (we could exit out of the evaluation loop, but it's easier to read if we don't). Then, we "and" the "clause satisfied" evaluation with the "equation solved" condition.

At the end of the evaluation loop, we check if we found a satisfiable assignment. If we have, we then make our *vars* field, which holds the solution if there is one, the

assignment, and then we return True to indicate that we have found a satisfiable assignment. If each possible assignment is not satisfiable, then we return False, and *vars* will hold nothing.

Since it's trivially easy to see why the code works, we will move onto something more interesting: optimizing brute force.

2.1 Optimizing Brute Force

I'll be completely honest with you, if you want to skip this section, go ahead. It's more or less just something interesting enough we can do to make the (arguably) worst algorithm[3] better.

Since we're dealing with pseudocode, we're not going to consider the hardware side of computers such as branch prediction, cache locality, or anything that the hardware uses to execute our code (other than the CPU). We will consider it from a pure Big-O perspective (excluding constant factors unless we're choosing between two algorithms with the same computational complexity).

The first thing we will add is short circuiting to our clause evaluation. After line 26 where we finish OR-ing the "clause satisfied" boolean, we will add in a new block:

```
1  ...
2  bool satisfied_clause = false;
3
4  for (j = 1 up to clause.size)
5  {
6      int variable = clause[j]
7      int v_idx = abs(variable)
8
9      if (variable > 0)
10     {
11         satisfied_clause =
12             Assignment[v_idx] || satisfied_clause
13     }
14     else
15     {
16         satisfied_clause =
17             !Assignment[v_idx] || satisfied_clause
18     }
19
20     if (satisfied_clause == True)
21     {
22         continue
23     }
24 }
25
26 ...
```

[3] I do not know of any algorithm worse than brute forcing.

This just says "once this clause is satisfied, go evaluate the next one immediately."

We can then add another conditional after this loop, which will move onto the next variable assignment the instant the equation is no longer satisfied:

```
...
bool solved = true;

for each (clause in clauses)
{
    bool satisfied_clause = false;

    for (j = 1 up to clause.size)
    {
        ...
    }

    if (satisfied_clause == True)
    {
        continue
    }

    solved &= satisfied_clause;

    if (solved == False)
    {
        break
    }
}

...
```

The next algorithm optimization we can make is removing the loop where we move the current assignment into the "Assignment" array. Rather than doing it upon every iteration, why don't we do it once we find a satisfiable assignment?

```
solveCNF()
{
    bool[1..numVars] Assignment

    for (i = 0 up to 1 << numVars)
    {
        bool solved = true

        for each (clause in clauses)
        {
            bool satisfied_clause = false

            for (j = 1 up to clause.size)
            {
```

```
15                  int variable = clause[j]
16                  int v_idx = abs(variable)
17
18                  if (variable > 0)
19                  {
20                      satisfied_clause = (i & (1 << j))
21                        || satisfied_clause
22                  }
23                  else
24                  {
25                      satisfied_clause = !(i & (1 << j))
26                        || satisfied_clause
27                  }
28
29                  if (satisfied_clause == True)
30                  {
31                      continue
32                  }
33              }
34
35              solved &= satisfied_clause
36
37              if (solved == False)
38              {
39                  break
40              }
41          }
42
43          if (solved == True)
44          {
45              for (j = 0 up to numVars - 1)
46              {
47                  Assignment[j] = i & (1 << j)
48              }
49
50              vars = Assignment
51              return true
52          }
53      }
54
55      return false
56  }
```

With that, that's all the optimization I can think of. In case you're wondering, no this does not change the asymptotic runtime of the method, however, it's easy to see why these changes would presumably make the algorithm run faster as we're adding code that stops evaluation the moment we've figured just enough out to know if it's worth continuing evaluation and deferring extra calculation (recording the current assignment) until once we know it's satisfiable.

As an exercise to the reader, I will ask you, how can you parallelize this algorithm? And once you've figured that out, can you program it in a way where once one thread finds a satisfactory assignment, it terminates all other threads immediately instead of waiting for the others to finish?

2.2 Brute Force - Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course.

To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here:

`https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`

We gave our current implementation 1000 SAT instances (uf20-91), each with 20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm **2,119,301 milliseconds**, which is roughly 35.32 minutes. This result is actually from the first time I wrote a brute force algorithm. The brute force result you can find on GitHub, it actually took **1,770,851 milliseconds**, which is about 29.51 minutes. I have no idea what I did in the rewrite that shaved off 4 minutes. I won't even lie to you reader, I am a busy undergraduate student, so I didn't even do the "best practice" for testing, which would be to quit all other processes and then benchmark it. However, while I was testing, I was doing a Canvas assignment, so I was doing lots of background work.

Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that we're going to be improving upon anyways.

Here are the results for all brute force algorithms, including the short-circuited and parallel versions:

```
On all 1000 equations in uf20-91:

Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).
```

For the parallel version, the implementation isn't perfect. I believe that we don't prevent false sharing amongst cache lines (ie we get more cache swaps between threads leading to more cache misses), which could explain why on my M1 MacBook Air, we don't get close to a 8 times speedup. Since this was for fun, I'm not going to perfect the parallelized version, that can an exercise to the reader :)

Chapter 3

DPLL

The Davis-Putnam-Logemann-Loveland (DPLL) algorithm is a complete backtracking based search algorithm for finding satisfiable assignments for SAT problems. It is, dare I say, the foundational algorithm for modern SAT solvers, as the state-of-the-art algorithm used by basically every SAT solver builds upon the foundations that DPLL provides, as we will see later on.
It consists of three components:

- Recursive backtracking
- Unit propagation
- Pure literal elimination

How does it work?

Imagine a binary tree of variables, each with two branches representing their assignment, one for true and one for false. Suppose we don't know anything about our CNF equation. That is, from looking at it, any variable could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable. How can we figure out if we can satisfy the equation without just brute forcing it?

The answer is ~~making an ass out of you and me~~ to make assumptions.

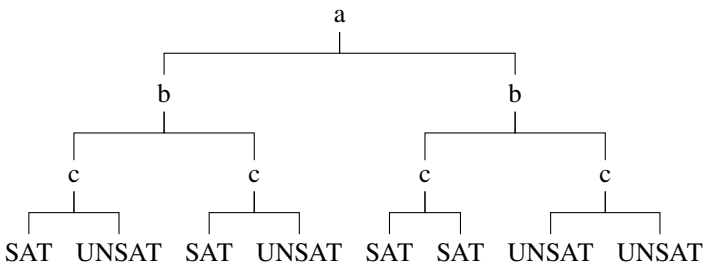


Figure 3.1: Example DPLL decision tree (the outcomes may not correspond to an actual CNF instance, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate. How does making assumptions help us? Do we just run a random number generator and if it returns 1, we go "okay, it's satisfiable," and false otherwise?

No. We make assumptions about our literal assignments.Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL we start with a literal, we'll call it A. Suppose there is nothing in the equation that indicates whether or not A should be true or false; either could be valid.

So to tackle our problem, initially we will assume A is true and see where that leads us. We propagate that value throughout our clauses, and suppose there is a clause that can't be satisfied. That's a contradiction, because if the equation was satisfiable then $A = True$ couldn't have led us there.

So we've figured out that at the very least $A = True$ leads to a contradiction. We've now eliminated half of the search space immediately and only need to consider the cases where $A = False$.

This may not make total sense immediately, so instead of giving one explanation of the algorithm and then building it, we will build up the algorithm and explain how each part works.

```
1
2
```

3.1 Recursive Backtracking

First, we need to rewrite our brute force algorithm to be recursive, as that's the foundation for the entire algorithm (duh). To do this, we will introduce a new method *solve()* which will be our recursive solving method. To also make things simpler, we will extract the equation evaluation into its own separate method as well.

This means our SAT solver becomes like so:

```
1  public interface:
```

[1]Note for future me: write a better introduction.

[2]`https://en.wikipedia.org/wiki/DPLL_algorithm`

```
2       SATSolver(string input)
3       bool solveCNF()
4
5   private interface:
6       void parse(string equation)
7       bool solve(bool[] assignment)
8       bool evaluate(bool[] assignment)
9       int numVars
10      int numClauses
11      int[][] clauses
12      optional bool[] lits
```

The flow essentially becomes like this: *solveCNF()* calls *solve()*, which calls itself over and over and returns whatever solution it finds back up to *solveCNF()*, who will handle any sort of cleanup.

Why can't we just make *solveCNF()* call itself recursively? Good question. That's because *solveCNF()* will handle any setup that *solve()* will need in order to function properly, and it will handle any cleanup that *solve()* may create.

Let's now discuss how *solve()* will work. It will be passed in a *bool[]* for the current assignment of literals. When the size of our assignment is equal to the number of variables, we will then evaluate the satisfiability of the assignment, and if it's satisfiable we return true. Otherwise, we return false, backtrack, retry, until we either find a satisfiable assignment or we've exhausted them all.

The pseudocode is as so, starting with *evaluate()*:

```
1   bool evaluate(bool[] assignment)
2   {
3       bool solved = true;
4
5       for each (clause in clauses)
6       {
7           bool satisfied_clause = false;
8
9           for (j = 1 up to clause.size)
10          {
11              int variable = clause[j];
12              int v_idx = abs(variable);
13
14              if (variable > 0)
15              {
16                  satisfied_clause = assignment[v_idx]
17                    || satisfied_clause;
18              }
19              else
20              {
21                  satisfied_clause = !assignment[v_idx]
22                    || satisfied_clause;
23              }
24
```

```
25              if (satisfied_clause == True)
26              {
27                  continue;
28              }
29          }
30
31          solved &= satisfied_clause;
32
33          if (solved == False)
34          {
35              break;
36          }
37      }
38
39      if (solved == True)
40      {
41          lits = assignment;
42      }
43      else
44      {
45          lits = no-value;
46      }
47
48      return solved;
49  }
```

As you can see, all we've done is move our equation evaluation from brute force into its own function. One thing to note is that this method also handles updating our *lits* field. Thus, if *evaluate()* returns true, then *lits* will hold the satisfying assignment, and if it returns false, then *lits* will hold nothing.

Now our recursive *solve()* method:

```
1   bool solve(bool[] assignment)
2   {
3       if (assignment.size == numVars)
4       {
5           return evaluate(assignment);
6       }
7
8       append(assignment, True);
9
10      if (solve(assignment))
11      {
12          return true;
13      }
14
15      assignment.back() = False;
16
17      if (solve(assignment))
```

```
18      {
19          return true;
20      }
21
22      truncate(assignment, 1);
23      return false;
24  }
```

This might be slightly confusing at first, so let's go over it.

We first check our base case. If our assignment has all of its variables, we evaluate it and return the result whether it be true or false. And you'll notice that whenever we return true from a call of *solve()*, we immediately return true afterwards, so therefore, if we find a satisfiable assignment, we will go up our call stack back to *solveCNF()*, so no need to worry that it will terminate if we find a satisfiable assignment.

However, if our assignment still needs some variables, we append true to our assignment, then we do a recursive call on *solve()*. If that recursive call leads to a satisfiable assignment, we return true, otherwise, we swap out our last value for false and try *solve()* again. And if that ends up working out, we return true, but if it doesn't then we remove the last value and return false.

So how do we know that it will guarantee termination for an unsatisfiable problem? Notice how when assigning the current variable true or false and it doesn't work, we just remove the last one and go up the call stack by one? Well suppose we're at the top of the call stack (we've assigned the first variable) and the first variable is assigned false, and that didn't lead to a satisfiable assignment. Then we remove that first variable from the assignment, and then we return false. So our assignment array is empty, and we've returned to *solveCNF()*. Thus, *solve()* will terminate eventually.

And how do we know that we'll try every single assignment? Well for the very last variable, it's easy to see, if it being true doesn't satisfy the equation, we assign it false, and if that doesn't work, we remove it and go up the call stack. Then for the previous variable, if it was true, we assign it false and go back to the last variable where it tries true and/or false again. Then that doesn't work, we go up the call stack twice, and that continues until we've found a satisfiable assignment or we've exhausted them all.[3]

Lastly, our *solveCNF()* method:

```
1   bool solveCNF()
2   {
3       bool[1..numVars] assignment;
4
5       return solve(assignment);
6   }
```

Because we're dealing with pseudocode, we don't have to do things such as reserving or resizing the assignment array. And since *evaluate()* takes care of making sure our *lits* field holds a valid assignment (if found) and holds nothing if not, we don't need to do anything else in *solveCNF()*.

[3]This is a very informal proof, but what's most important is that we build up our understanding of what's going on instead of dumping notation. Perhaps I will do a formal proof in the future.

You may wonder though, why not pass in *lits* to *solve()* or have no setup and make *solveCNF()* purely recursive? Those are very valid questions to ask.

This has to do with the differences between pseudocode and actual implementation. In the actual implementation, some methods depend on the state of *lits*, such as whether or not it holds anything, and by passing it into *solve()*, we force it to hold something, even if there is no satisfiable assignment. And it's true, that in *solveCNF()* if we had no setup, we could make it recursive on *lits*, however, I chose to implement it differently.

When you are reading this, please ask yourself these kinds of questions, because oftentimes, the way you read my implementation is not the best way, and even if it is, there is no reason as to why you can't think of different implementations, try them out yourself, and compare. Perhaps, and I expect this will happen often, you will find a better implementation than the one written down here.

3.1.1 Recursive Backtracking – Results

Running this code on the same 1000 instances (uf20-91), it took **1,846,644ms**, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm. The next time I rewrote this algorithm, it took **1,540,613ms**, which is about 25.68 minutes. So once again, that's about a 5 minute speedup just from rewriting it. Perhaps we should rewrite every piece of software in the world to make it run 5 minutes faster.

Here's a question for you, why might assigning our variables true instead of false first lead to an increase in speed? **Hint: Look at the number of negations present in the testing set.**

```
On all 1000 equations in uf20-91:

Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).
```

3.2 Unit Propagation

Unit propagation is the next step in implementing DPLL, and it introduces the idea of implication. Imagine a clause with two variables: A and Z, and we know that $A = False$. What can we say about Z? Well, if our equation is satisfiable, then the clause must be satisfiable, therefore $Z = True$ because if it didn't, that would be a contradiction.

So instead of assigning variables $B, C, \ldots, Y$, then assigning Z, why don't we just do it now, and use that knowledge in other clauses containing Z or $\neg Z$?

That is essentially the idea behind unit propagation: the decisions we make imply other assignments, and so we should force them now instead of waiting until later.

However, this introduces a light hiccup into our implementation. You see, our initial invariant about assigning variables is that they are assigned in numerical order instead of (possibly) out of order.

"Not a problem! We'll just make the assignment array have space for every variable so we can index it anywhere," I hear you think. Unfortunately that interferes with our base case because it relies on the size of the assignment array. If it is initially at a fixed size of *numVars*, then our base case will always fire because our assignment array always has a size of *numVars*. So we would either need to change our base case or change how we store learned variables.

What about a learned set? One that holds whether or not we've learned a variable (positive or negative), and then as we traverse down the call stack, we just check if a variable is already learned, and if so, we give it its learned value?

That would 100% work. In fact, the first time I implemented DPLL, that is exactly what I used. However, it has the disadvantage of being annoying to maintain. Instead, why don't we create a new data type called something like *var* that represents the state of each variable: *True*, *False*, and *Unassigned*.

Then we can just maintain a simple counter of how many are not *Unassigned*, and then that keeps our base case nice and neat. Yes dear reader, we are going to do that, however, if you'd like to use the learned set, reading about how we do it without a learned set will help you and offer clues on how to implement it your way.

3.2.1 Changes To Our Implementation

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
13        void parse(string equation)
14        bool solve(var[] assignment, int level)
15        bool evaluate(var[] assignment)
16        void propagate()
17
18        int numVars
19        int numClauses
20        int numAssigned
21
22        int[][] clauses
23
24        optional var[] vars
```

Here we just add in a new enum for the states that a variable can be, and adjust the methods using *bool*s to use the new enum type instead.

3.2.2 “Textbook" Unit Propagation

Before we start changing our algorithm, let's implement the *propagate()* method.

As stated before, the idea is that given a clause with either only one variable or only one *Unassigned* variable (and the rest evaluate to *False*), we can immediately say that *Unassigned* variable will be whatever value it needs to be to evaluate to *True* for that clause.

In the “textbook" way of implementation, [4] once a clause is satisfied because of a literal, you remove every other clause that is satisfied by that literal. And for every other clause containing the negation of the literal, you remove that negation entirely. This is done to shrink the search space.

We will implement this algorithm with as much malicious compliance as you can carry at an office job without getting fired:

```
1     bool solveCNF() {
2         var[1..numVars] assignment
3
4         for each (var in assignment) {
5             var = Unassigned
6         }
7
8         propagate(assignment)
9         solve(assignment, 0)
10    }
```

We want to propagate before we start solving in case there are clauses with only one variable (it's already unit).

```
1     bool solve(bool[1..numVars] Assignment, int level) {
2         if (level == numVars - 1) {
3             return evaluate(Assignment)
4         }
5
6         bool[1..N] backupAssignment = assignment
7         int[][] backupClauses = clauses
8
9         if (Assignment[level] == Unassigned) {
10            Assignment[level] = True
11            propagate(Assignment)
12        }
13
```

[4] https://en.wikipedia.org/wiki/Unit_propagation

```
14          if (solve(Assignment, level + 1)) {
15              return True
16          }
17
18          clauses = backupClauses
19          Assignment = backupAssignment
20
21          if (Assignment[level] == Unassigned) {
22              Assignment[level] = False
23              propagate(Assignment)
24          }
25
26          if (solve(Assignment, level + 1)) {
27              return True
28          }
29
30          return False
31      }
```

Before we get onto the code for unit propagation, let's understand this code first. Because unit propagation no longer guarantees that variables are assigned in order, we need to pass in a "level" as a parameter so we know which variable we need to assign. We also now need to check that propagation didn't already assign this variable. If it's currently Unassigned, we need to assign it then propagate, but if it's been assigned before we assign it, then we know that it's the result of propagation, so we can continue onto the next level. Other than that, it's practically the same as our recursive backtracking code, except now we need to have a "backup" of our clauses and our assignment.

Why? Because *propagate()* **mutates** the clauses and the assignment in a way that breaks the assumption that our literals are assigned in "numeric" order. Since it returns nothing, we don't have a way to undo that easily, so instead, for readability, we just have a backup copy and whenever propagation leads to an unsatisfying assignment, we return false and undo whatever changes we made by copying back into our solver's fields. And the reason why we don't need to undo the propagation before returning False is because when we move back up the call stack by one level, the method continues by undoing the effects for us. Now let's look at how we perform textbook unit propagation:

```
1       void propagate(int[1..N] Assignment) {
2           while (true) {
3               Set<int> unitClauses;
4               Set<int> unitVars;
5
6               for (i = 0 up to numClauses) {
7                   clause = clauses[i]
8
9                   bool isUnit = True
10                  int unassignedIdx = -1
11                  int numUnassigned = 0
12
```

```
13                  for (j = 0 up to clause.size) {
14                      int var = clause[j]
15                      int idx = abs(var)
16
17                      if (Assignment[idx] == Unassigned) {
18                          numUnassigned = numUnassigned + 1
19                          unassignedIdx = j
20                      } else if (var > 0 && clause[j] == True
21                          || var < 0 && clause[j] == False)
22                      {
23                          isUnit = false
24                      }
25                  }
26
27                  isUnit = isUnit && numUnassigned == 1
28
29                  if (isUnit) {
30                      add(unitClauses, i)
31                      add(unitVars, clause[unassignedIdx])
32
33                      int var = clause[unassignedIdx]
34                      int idx = abs(var)
35
36                      if (var > 0) {
37                          assignment[idx] = True
38                      } else {
39                          assignment[idx] = False
40                      }
41                  }
42              }
43
44              int clauseIdx = 0;
45              for each (clause in clauses) {
46                  for each (var in clause) {
47                      if (contains(unitVar, -var)) {
48                          remove(clause, -var)
49                      }
50
51                      if (contains(unitVar, var)
52                          && contains(unitClause, clauseIdx))
53                      {
54                          remove(clauses, clause)
55                      }
56                  }
57
58                  clauseIdx = clauseIdx + 1
59              }
60
61              if (unitClauses.size == 0 && unitVars.size == 0) {
62                  break
```

```
63              }
64          }
65      }
```

Okay, now let's understand what's happening here. We immediately go into an infinite loop (nice), and we start looking at each clause for two things:

1. The clause has only one Unassigned variable
2. Every other literal in the clause is False (remember, literals are evaluated variables)

If both things are true, then the Unassigned variable will be assigned whatever value is required to make it evaluate to True within that clause, and we know we have found a unit variable and a unit clause. After we have found all of our unit variables and unit clauses, we then rescan every other clause and delete the each unit variable's negative, (if we learn that -5 is unit, then we remove +5), and if the clause contains a unit variable, we just delete that clause entirely from the database. Once we stop finding new unit variables and new unit clauses, we stop entirely. You may wonder (I reread my code and thought I found a bug), "how do we know that we always terminate?" That's a great question. You see, this confusion may come from the fact that we rescan the database every iteration of the *while()* loop searching for unit clauses. What is stopping us from re-finding the same clause as unit upon a new iteration? The answer is much simpler than you'd expect: since we assign our unit variables, the clauses upon the next iteration no longer foloow our criteria for being unit propagate-able because there is now a literal in that clause that is True. Therefore, the number of unit variables and clauses monotonically decrease between iterations (a fancy way of saying the number can only go down).

Now, you may be wondering, "what's the performance of our solver now?" Let me answer that as plainly as possible: worse than brute force. How? Easy! We are doing SO MANY things that hurt the computer. We are allocating TONS of memory in order to backtrack, we are allocating even more memory to propagate (hash sets aren't cheap), and hashing, while O(1), is not a cheap operation to do. So yeah, I can believe that it's worse than brute force because at least the brute force algorithm was easy on the CPU. Let's learn how this is *usually* [5] implemented!

3.3 Better Unit Propagation

So the textbook style unit propagation sucked. Let's change that. As discussed before, the biggest bottleneck/inefficiency in the actual implementation is the number of copies we need to make of our clauses and assignments on each decision level. Assigning one variable creates a new decision level, so 20 variables means 20 levels. For an unsatisfiable CNF equation, I estimate we need to make about 3 copies of our clauses

[5] I wanted to say "actually implemented", however, that's a big generalization, and again, since I'm not the expert here, I don't think I have the authority to say that. The next section is how *I* implemented it, and it roughly follows the idea on how SAT solvers implement this.

and assignments: one for the initial storage, another for the first failure, and a third for the second. Even if we ignore the inefficiencies in how we propagate (memory allocations, traversals, and adjustments our arrays would need to make when we delete things), this is a huge amount of memory to store. If you think that 20 variables and 91 clauses is a lot, modern SAT solvers can handle equations with **millions** of variables, so creating an algorithm that makes brute force look attractive is actually an achievement. [6]

So what's the strategy? Firstly, it would be nice if we didn't actually mutate our clauses at all. Ie, no clause deletion, and no variable deletion. Next, it would be nice if we didn't need hash sets in order to store our unit variables. Instead, it would be nice if we had a data structure where we put in our decided variables, and then it slowly shrinks as we iterate through. Ie, we propagate one on variable at a time and never again. This sounds like we need a *queue*. [7]

Lastly, instead of copying our assignment, and constantly reiterating through the *entire* clause database, what if we just marked which variables we assigned, and which clauses we've satisfied? That way when we need to undo, we just mark those variables as Unassigned, and those clauses as no longer satisfied. That way we save a lot of memory (and besides, copying is NOT O(1) as you might be led to believe by that deceptively simply "=" operator).

Let's change our interface:

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
13        void parse(string equation)
14        bool solve(var[] assignment, int level)
15        bool evaluate(var[] assignment)
16        (int[] vars, int[] clauses) propagate()
17
18        int numVars
19        int numClauses
20        int numAssigned
21
22        int[][] clauses
23        bool[] satisfied
24
```

[6] They might revoke my degree for that actually

[7] That wasn't shoehorned in at all!

```
25          optional var[] vars
```

We need to make some minimal changes to *solveCNF()*:

```
1       bool solveCNF() {
2           var[1..numVars] assignment
3
4           for each (var in assignment) {
5               var = Unassigned
6           }
7
8           propagate(assignment, 0)
9           solve(assignment, 0)
10      }
```

Since *propagate()* now takes a decision level/variable index, we need to pass one to the method. Although no variable has an index of 0 (because it's reserved to mark the end of a clause), passing in 0 just tells it "Hey, there isn't a literal to propagate on, so just find all clauses with only one variables and mark them as satisfied and 'learn' those variables." And since this is at the root level, we don't need to undo anything if it turns out that the equation is unsatisfiable, since we're gathering things that HAVE to be a certain value if this equation is to be satisfiable.

Since we don't need to change *solveCNF()*, we can go straight to *solve()*:

```
1       bool solve(int[1..N] Assignment, int level) {
2           if (level == numVars - 1) {
3               return evaluate(Assignment);
4           }
5
6           if (Assignment[level] != Unassigned) {
7               return solve(Assignment, level + 1)
8           }
9
10          Assignment[level] = True
11          (int[] unitVars, int[] satisfiedClauses)
12              = propagate(Assignment, level)
13
14          if (solve(Assignment, level + 1) == True) {
15              return True
16          }
17
18          for each (var in unitVars) {
19              int idx = abs(var)
20              Assignment[idx] = Unassigned
21          }
22
23          for each (clauseIdx in satisfiedClauses) {
24              satisfied[clauseIdx] = False
25          }
```

```
26
27          Assignment[level] = False
28          (int new_unitVars, int new_satisfiedClauses)
29              = propagate(Assignment, level)
30
31           if (solve(Assignment, level + 1) == True) {
32              return True
33          }
34
35          for each (var in new_unitVars) {
36              int idx = abs(var)
37              Assignment[idx] = Unassigned
38          }
39
40          for each (clauseIdx in new_satisfiedClauses) {
41              satisfied[clauseIdx] = False
42          }
43
44          Assignment[level] = Unassigned
45          return False
46      }
```

This method is practically the same, except now, we just check if we've already assigned the variable at the current decision level. If we have assigned a variable before we reach its decision level, then it must have been inferred/learned from the *propagate()* method, so we don't need to call it again. Then, it's business as usual: assume the current variable is True, propagate, if it leads to a satisfiable assignment, return True, otherwise, we undo what we've assigned and marked as learned, then repeat with False.

What's more interesting is our shiny new *propagate()* method:

```
1       (int[] vars, int[] clauses) propagate(int[] assignment, int level
        ) {
2           queue<int> unit
3
4           int[] unitVars
5           int[] satisfiedClauses
6
7           unit.enqueue(level)
8
9           while (unit.size > 0) {
10              unit.dequeue()
11
12              for (i = 0 up to numClauses) {
13                  if (satisfied[i]) {
14                      continue
15                  }
16
17                  clause = clauses[i]
18
```

```
19                  bool isUnit = True
20                  int unassignedIdx = -1
21                  int numUnassigned = 0
22
23                  for (j = 0 up to clause.size) {
24                      int var = clause[j]
25                      int idx = abs(var)
26
27                      if (Assignment[idx] == Unassigned) {
28                          numUnassigned = numUnassigned + 1
29                          unassignedIdx = j
30                      } else if (var > 0 && clause[j] == True
31                          || var < 0 && clause[j] == False)
32                      {
33                          isUnit = false
34
35                          satisfied[i] = true
36                          append(satisfiedClauses, i)
37                      }
38                  }
39
40                  isUnit = isUnit && numUnassigned == 1
41
42                  if (isUnit) {
43                      add(unitClauses, i)
44                      add(unitVars, clause[unassignedIdx])
45
46                      int var = clause[unassignedIdx]
47                      int idx = abs(var)
48
49                      if (var > 0) {
50                          assignment[idx] = True
51                      } else {
52                          assignment[idx] = False
53                      }
54
55                      unit.enqueue(var)
56
57                      append(unitVars, var)
58                      satisfied[i] = true
59                      append(satisfiedClauses, i)
60                  }
61              }
62          }
63
64          return (unitVars, satisfiedClauses)
65      }
```

So, let's discuss what's going on. *propagate()* now takes in the decision level, which

is also the same thing as an index for a variable (conceptually speaking). It then adds that to a queue, then while the queue is not empty, it scans the clauses as before. First it makes sure we're not operating on a clause that's already been satisfied (because we can't tell anything from it), and when it finds a clause that has a true literal, it marks the clause as satisfied. Then when it finds a unit clause, it marks the clause as satisfied, keeps track of the new unit variable its found, and then adds that unit variable to the queue. Then once it's done, it returns the unit variables and the indices of the satisfied clauses.

You may have some questions about this approach. Firstly, how do we make sure we don't ever add the same variable twice? The answer is the same as before: because we assign a variable once it's found to be in a unit clause, it can't be found as Unassigned again, and therefore can't be marked as a unit variable again. Then, since we skip over already satisfied clauses, the amount of clauses we scan each iteration decreases monotonically.

And going back to our *solve()* code, even if we mark the same clause as satisfied or the same variable as unit twice, it doesn't matter because when we backtrack, we simply restore it back to its state of unsatisfied or Unassigned, respectively. What's important is that we don't mark the same clause or variable twice *across decision levels* because then, and only then, would our backtracking code become a problem because we wouldn't be correctly restoring the state of the equation back to what it was before the decision level was entered. But since we can't mark the same variable or clause twice **until** we backtrack, there's nothing to worry about.

Before we end this section, you may also be wondering: can we use the *satisfied* array in our *evaluate()* method to skip clauses we know we've already satisfied? And the answer is yes, and I would encourage you to do so in your implementations.

3.3.1 Better Unit Propagation - Results

So after implementing better unit propagation, this took **868ms** to solve all 1000 satisfiable equations. The first time I implemented this version of unit propagation, it actually took **2363ms**, so this is somehow over twice as fast. And because it's so fast, just like the first time I implemented this, we will be now testing our solver on unsatisfiable equations as well.

Our second testing set comes from the same database as before. [8] And our next testing set is called "UUF50.218.1000." In other words, it's full of equations that have 50 variables, 218 clauses, and there are 1000 such equations in the set. In order to solve all 1000 equations, it took **140,411ms** (2.34 minutes). That's also much faster than the first time I implemented this algorithm. The first time I did, it took 646,228ms (10.77 minutes). Honestly, with this much speedup, I don't know if I was just bad at implementing these algorithms the first time, [9] or if downgrading my operating system (and subsequently wiping my harddrive clean) [10] just has that much of an effect on execution speed. [11]

[8] `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`, in case you needed a reminder.
[9] Highly likely
[10] I HATED MacOS Tahoe. Sequoia is where it's at.
[11] Also likely, depending on how much RAM and swap is being used to begin with.

Why didn't we test our previous versions out on unsatisfiable equations? That's a good question. For brute forcers, since there is no satisfiable assignment, it needs to check every single one to conclude that it's unsatisfiable, which would take a lot of time. Likewise with our recursive backtracking solver. And since our (my) implementation of textbook unit propagation was *worse* than brute force, [12] obviously we wouldn't want to test it on something that will take more time than a satisfiable equation. Also because the asymptotic runtime is *still* exponential, having a much larger equation would take a much, much larger amount of time, even if they were satisfiable.

```
Results so far:

On all 1000 equations in uf20-91:

Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Unit Propagation: 140,041ms (~2.34 minutes).
```

3.4 Pure Literal Elimination

Suppose I gave you the following CNF equation:

```
1 2 -3 0
1 -4 5 0
1 -2 5 0
```

Do you notice anything odd about it? Perhaps "odd" is not the correct word here, do you notice an easy way to satisfy this equation? You probably noticed that the variable 1 shows up in every clause and is always in the positive. So, if you just assign $1 = T$, then it doesn't matter what the rest of the variables are assigned, because $1 = T$ satisfies every clause! This is idea behind **Pure Literal Elimination**. Essentially, every now and then, we scan our equation for "pure literals", which are variables that only show up in the positive or negative (but never both), and then give them whatever assignment is necessary to have their literals evaluate to True.

Now, suppose I give you this equation:

```
1 0
-1 -2 -3 0
1 2 0
-2 3 0
```

[12] A difficult feat.

I have a question for you dear reader: is -2 a pure literal? You might be inclined to say “no" because -2 and 2 are both present in the equation. However, despite that, the answer is “yes.” Why? Well notice in the first clause that 1 is a unit variable, so $1 = T$ is forced by pure literal elimination. Then the third clause, 120, because it's already satisfied, the variable 2 doesn't matter anymore. The other two clauses, which are unsatisfied, are the ones that matter. They both contain -2, and because they're unsatisfied, we can force $2 = F$ to satisfy them, thus making the assignment of 3 irrelevant.

Okay, that makes sense, sort of. How come we're allowed to do that? Like, how do we know that forcing $2 = F$ wouldn't screw up the equation elsewhere (if it were larger)? And the answer is: because it doesn't hurt the other clauses. In the third clause, once it's satisfied, the only thing that can make it unsatisfied is the backtracking, but since 1 is unit, there shouldn't be any backtracking that would suddenly make its satisfiability rest on the assignment of 2. Likewise, for the other two clauses, no matter what value 3 takes, $2 = F$ must be true to satisfy the other clause (try it out for yourself!), so we can go straight ahead and assign $2 = F$ and skip deciding.

Okay, sure, but there are 8 total assignments that we can get. How do we know that forcing $2 = F$ would lead to a satisfiable assignment? As in, how do we know we're not prematurely boxing ourselves in? That's another great question! The reason why we're not boxing ourselves in prematurely is because, again, it only helps the satisfiability of the whole equation, and because we're not doing anything detrimental, we can make that decision. [13]

3.4.1 Implementing Pure Literal Elimination

Now that we (sort of) understand why it works, how do we implement it? We need some form of efficient tracking for things we've already encountered, perhaps something like a map or a set. Then, we just need to make sure we've only encountered one version of a literal, and if we have, then we know it's pure. Then we just need to gather our pure literals and the satisfied clauses, and then return them.

```
1  (int[] var, int[] clauses) pureElim(int[] assignment) {
2      map<int, int[]> var_to_clause
3      int[] pure
4      int[] satisfied
5
6      for (i = 0 up to numClauses) {
7          if (satisfied[i]) {
8              continue
9          }
10
11         clause = clause[i]
12         for each (var in clause) {
13             append(var_to_clause[var], i)
14         }
```

[13] Essentially, think of it as making the solution “more correct” by forcing the pure literal.

```
15      }
16
17      for each ([var, clauses] in var_to_clause) {
18          idx = abs(var)
19
20          if (contains(var_to_clause, -var) or assignment[idx] !=
        Unassigned) {
21              continue
22          }
23
24          if (var > 0) {
25              assignment[idx] = True
26          } else {
27              assignment[idx] = False
28          }
29
30          for (i = 0 up to clauses.size) {
31              satisfied[clauses[i]] = true
32              append(satisfied, clauses[i])
33          }
34
35          append(pure, var)
36      }
37
38      return (pure, satisfied)
39  }
```

So this is a single pass version of pure literal elimination. It is possible that after finding some pure literals, and marking some clauses satisfied, a second pass could find new pure literals. So why are we implementing a single pass version? Besides it not being a technique used in modern SAT solvers, in my experience, which isn't much since I'm being honest, it doesn't make a big (positive) difference in solve time efficiency. And besides, if we wanted a multi-pass version, we could wrap it in *while(true)* loop and break once we find no more new pure literals.

Now let's get into how the algorithm works. We first build a map from variables to their clauses (in unsatisfied clauses). Next, afterwards, we then check for each element in the map, that the map does not contain the negative variable of the current entry, and if it doesn't, we then need to make sure that the current variable is Unassigned. If the current entry passes both those conditions (pure and Unassigned), we assign it so it evaluates to True. Then, we mark each clause the variable is found in as satisfied, and then we add the indices of the satisfied clauses into the return values.

3.4.2 Pure Literal Elimination - Results

So. I have some terrible news. After implementing pure literal elimination, our solving time ***increased***. I know, I know. It's not supposed to happen, but it can happen. There are many possible explanations for this. For instance, pure literal elimination takes a decent amount of work to perform. We have to scan all unsatisfied clauses, and since

there is no way to determine if a literal is impure until after everything is scanned (we could have used a hash set, but then we'd need to hash for every literal), we're accruing memory and runtime costs. Second, even if pure literal elimination finds something, it usually isn't much (at least for the testing sets currently in use), so we're doing so much work for so little in return. Third, it increases our backtracking cost because now we need to undo unit propagation and the pure literal elimination. Fourth, my implementation isn't perfect. It is possible that forcing a pure literal may force another pure literal in a different clause, so you would need multiple passes to do. However, our implementation is just one pass (wrap it in a *while* loop or something). [14]

However, no one cares *why* something performed badly. People only care about *how* badly something performed. So here are the results: For uf20.91: 1548ms, almost a 2x increase from just unit propagation. For UUF50.218.1000: 231,107ms (3.85 minutes), about a 1.5 minute increase from just unit propagation.

```
On all 1000 equations in uf20-91:

Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

Pure Literal Elimination had about a **2x** increase (which is bad) in solving time when compared to just Unit Propagation on uf20.91, which is the test set of satisfiable problems. It also had about a **1.6x** increase in solving time on UUF50.218.1000, which is the set of unsatisfiable problems.

3.5 Watched Literals

Before we get into the Watched Literal algorithm, I want to ease us into it. Why? Because it's so different from our current version of unit propgation. For a moment (and for the rest of the book) let's forget about pure literal elimination; it wasn't that great anyways. Unit propagation gave us a massive speedup oppossed to recursive backtracking, taking runtimes down from minutes to seconds and from half-hours to mere minutes. Despite all of that, it's really inefficient in the sense that we're going so much work for so little in return.

[14] Modern SAT solvers don't tend to use pure literal elimination for reasons outside the "it wasn't very helpful" aspect, and next chapter we'll see why.

Let's define what a **contradiction** is, because we've been using that word here and there, but beyond "unsatisfiable," we haven't formally defined what it means. A **contradiction** is the result of a literal forcing a clause to evaluate to False, leading to the whole assignment to become unsatisfiable. You might remember how in the brute force solver, we added an *if()* statement at the end of evaluating a singular clause that checked if that clause evaluated to False. If it did, we skipped that assignment and moved onto the next one, but if it didn't, we continue our evaluation.

This led to a pretty significant speedup in the brute forcer because instead of evaluating everything then deciding, we did it incrementally, allowing us to quickly leave when something was bad. At the moment, our original idea for unit propagation follows that first idea of delaying evaluation to the end. However, if you go back and look at the pseudocode, you'll notice that we do something *almost* the same as checking if a clause evaluates to False, at the moment, we check if everything except for one element evaluates to False, and that one element is Unassigned (since we made it so that unit propagation only operates on unsatisfied clauses). However, notice that when a clause evaluates to False, rather than stopping our propagation and assignment, we continue until we reach the evaluation phase?

Just like in brute force, the idea is the same: once a clause evaluates to False, no matter where we are in our assignment, there's no point in continuing on because we've found out that nothing will unfalsify that clause. As an exercise, I want you to try adding that into your unit propagation code and seeing how fast it takes to run on our test sets. I would do it, but I have a massive headache from implementing this algorithm last night. [15] Once you're done, we'll discuss watched literals some more.

So that early contradiction detection is one of the ideas behind watched literals. It's not one the key ideas, but it is part of it. There are some other ideas behind watched literals. For instance: zero cost backtracking. Not O(1) backtracking, 0 backtracking. Speaking of cost, do you notice something else wasteful about the way we propagate? If not for our *satisfied* marker array, we'd have to rescan the whole equation every time we propagated, which is about twice per level. That's pretty wasteful. After all, why do have to scan every clause in their entirety every single time? Especially because, usually, clauses don't become unsatisfied very often. And because they're not at risk of becoming unsatisfied very often, why do we need to scan it? And besides, it's not like every single variable appears in every single clause, so why not scan only the clauses we have to? I hope these questions are leading you to think of other questions regarding inefficiencies in our current implementation.

These questions were the driving force into the invention of the *Watched Literals* algorithm.

INSERT HISTORY, WHICH IS INTERESTING BTW, HERE [16]

Let's learn how the Watched Literals algorithm works. Earlier, I made a comment about how usually, each clause doesn't contain every single variable in the equation. So when we force a variable's assignment, we don't need to check every single clause, only at most the ones that contain the variable and its negation. (Ie, if we force a variable, say

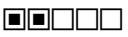
[15] This headache was also induced by my lack of ability to fall asleep at all last night AND a really intense session at the gym. Computer science gives you headaches for sure, but not like this one.

[16] `https://school.a4cp.org/summer2011/slides/Gent/SATCP3.pdf`

$2 = T$, we only need to check the clauses containing 2 and the clauses containing -2). However, do we *need* to check the clauses containing 2? Think about it. Those clauses are already satisfied by $2 = T$, so there's nothing to propagate on, for those clauses. The only clauses that have a *chance* to produce a propagation are the ones containing -2 because $-2 = F$, so if they have one Unassigned variable and everything else is now False, we can propagate!

3.5.1 A Visual Example of Watched Literals

Let's view a visual example. Suppose we had a clause with 5 literals (doesn't matter what they are). The first two literals are watched



$UUUUU$

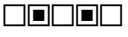
Suppose the first literal now evaluates to False. We now scan the clause for another literal that doesn't evaluate to False. We will find it at the third literal.

$FUUUU$

Now suppose the fifth literal becomes False. We do nothing because we're not watching the fifth literal.

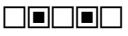
$FUUUF$

Now the third literal becomes False. We scan again for another not-False literal.



$FUFUF$

Now the second literal becomes False. Other than the second watched literal (the fourth), there are no more not-False literals. We don't relocate the watch.



$FFFUF$

However, now the second watched literal knows it's the last Unassigned literal in the clause. It will now evaluate to True!

$FFFTF$

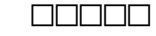
Our clause is now satisfied. Suppose instead, when we were here:

$FFFUF$

That the fourth literal also became False.

$FFFFF$

We've now encountered a *contradiction*, meaning that whatever assignment we have is impossible to satisfy the equation. Suppose we backtrack, and that restores us to a state where two literals in this same clause are now Unassigned. What happens to the watched literals?



$UUFFF$

The answer is that they stay where they are.

$UUFFF$

This is because one watched literal is Unassigned, so everything is okay. Although this last diagram isn't accurate to what a real restored state would look like, we can suspend our disbelief for the sake of simplicitly. The reason why watched literals work is because of the **Watched Literal Invariant**, which is that at any given point in time, at least one of the watched literals is either True or Unassigned. This invariant is what allowed us to propagate the clause when there was only one Unassigned literal left, and we will see further how it helps.

3.5.2 Algorithmic Changes

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      enum var
7      {
8          False
9          True
10         Unassigned
11     }
12
13     void parse(string equation)
14     bool solve(var[] assignment, int level)
15     optional int[] delta propagate(var[] assignment)
16     bool createWatched()
17
18     int numVars
19     int numClauses
20
21     int[][] clauses
22     bool[] satisfied
23     queue<int> unitProp
24
25     optional var[] vars
26     map<int, int[]> var_to_clause
27     map<int, (int, int)> clause_to_vars
```

So firstly, there are some major differences. We removed the *evaluate()* method. This is because now that propagation checks for contradictions, having all variables assigned is now ***equivalent*** to finding a satisfactory assignment of variables. So we don't need to evaluate our assignment anymore! That's significantly less work than what we had before, which is what we want. Next, we added in a queue, *unitProp*, kind of like what we had for unit propagation before. This time, it's now a member of the solver object instead of a local object within the method. We will get back to that in a second. Lastly, we added in two maps, one from variables to clauses, and the other from clauses to a pair of variables.

These data structures are used for fast lookup between variables to ***indices*** of clauses (for *var_to_clause*), and one from ***indices*** of clauses to ***indices*** of watched literals. It's important that we remember we're using indices instead of actual clauses because forgetting how we're using our mapping is going to create a lot of bugs and we're gonna have a bad time.

We also added in a new method, called *createWatched()* and what it does is it creates our initial maps for var_to_clause and clause_to_vars. Here is the pseudocode:

```
1       bool createWatched(int[] assignment) {
2           for (i = 0 up to numClauses) {
3               clause = clauses[i]
4
5               if (clause.size >= 2) {
6                   append(var_to_clause[clause[0]], i)
7                   append(var_to_clause[clause[1]], i)
8
9                   clause_to_var[i] = {0, 1}
10              } else if (clause.size == 1) {
11                  int var = clause[0]
12                  int idx = abs(var)
13                  append(var_to_clause[var], i)
14                  clause_to_var[i] = {0, 0}
15
16                  enqueue(unitProp, var)
17
18                  if (var > 0) {
19                      assignment[var] = True
20                  } else {
21                      assignment[var] = False
22                  }
23              } else {
24                  return False
25              }
26          }
27
28          return True
29      }
```

So here's how to read it: for each clause, it will fall into one of three cases:

1. It's size is greater than or equal to 2
2. It only has 1 variable
3. It's empty

In the first case, the two literals that will be watching that clause will be the first two literals. So the index i gets appended to their array of clause indices. Then, we record that clause i is being watched by its first and second literal.

In the second case, since there's only one literal, it gets watched by only that literal (duh). We also record that clause i is being watched by only its first literal. And lastly, we enqueue that literal to our unit propagation queue so we can do a root level unit propagation before we start solving. We also want to immediately force the assignment of the variable so that way we can use its value immediately in solving.

In the last case, since there are no literals, this clause is automatically unsatisfiable, so therefore this whole equation is unsatisfiable, so we return False so that when we call this method, we know that we can't solve it.

Now let's look at our modified solving methods:

```
1     bool solveCNF() {
2         var[1..numVars] assignment
3
4         for each (var in assignment) {
5             var = Unassigned
6         }
7
8         if (createWatched(assignment) == False) {
9             return False
10        }
11
12        if (propagate(assignment, 0) returns nothing) {
13            return False
14        }
15
16        solve(assignment, 0)
17    }
```

There are some slight differences: we create our watched literals and make sure that there isn't an unsatisfiable clause, and we make sure that *propagate()* returns something. If both those things are successful, then we can start solving away.

```
1     bool solve(int[] assignment, int level) {
2         if (level == numVars) {
3             return True
4         }
5
6         if (assignment[level] != Unassigned) {
7             return solve(assignment, level + 1)
8         }
9
10        int cur_var = level + 1
11
12        assignment[level] = True
13
14        optional int[] prop_result = propagate(assignment, cur_var)
15
16        if (prop_result is optional) {
17            assignment[level] = False
18
19            optional int[] new_prop_result
20                = propagate(assignment, -cur_var)
21
22            if (new_prop_result is optional) {
23                return False
24            }
25
26            if (solve(assignment, level + 1)) {
27                return True
```

```
28              }
29
30              int[] new_delta = new_prop_result.value
31
32              for each (int var_idx in new_delta) {
33                  assignment[var_idx] = Unassigned
34              }
35
36              return False
37          }
38
39          if (solve(assignment, level + 1)) {
40              return True
41          }
42
43          int[] delta = prop_result.value
44
45          for each (int var_idx in delta) {
46              assignment[var_idx] = Unassigned
47          }
48
49          assignment[level] = False
50
51          optional int[] new_prop_result
52              = propagate(assignment, -cur_var)
53
54          if (new_prop_result is optional) {
55              return False
56          }
57
58          if (solve(assignment, level + 1)) {
59              return True
60          }
61
62          int[] new_delta = new_prop_result.value
63
64          for each (int var_idx in new_delta) {
65              assignment[var_idx] = Unassigned
66          }
67
68          return False
69      }
```

This is almost the same as our old unit propagation code, except for that we don't keep track of which clauses are satisfied across levels, and the propagation method returns an array of variable indices instead of the actual variables. The code is also split up into multiple branches depending on whether or not propagation was successful. Some parts of the code seem really similar to each other because of this. I am sure that there may be a way to simplify the similar parts, but I like the way I wrote it because I

find it readable and the first time I wrote it, it was very nested.

Now for the interesting part:

```
1     optional int[] propagate(int[] assignment, int decision) {
2         int[] delta
3
4         if (decision != 0) {
5             enqueue(unitProp, decision)
6
7             int idx = abs(decision)
8             append(delta, idx)
9         }
10
11        while (unitProp.size != 0) {
12            int prop = unitProp.pop()
13
14            if (contains(var_to_clause, -prop) == False) {
15                continue
16            }
17
18            int[] watched_clauses = var_to_clause[-prop]
19
20            for (i = 0 up to watched_clauses.size) {
21                clause_idx = watched_clauses[i]
22
23                int[] clause = clauses[clause_idx]
24                (int, int) watches = clause_to_var[clause_idx]
25
26                if (clause[watches.first] != -prop) {
27                    swap(watches.first, watches.second)
28                }
29
30                bool relocated = False
31                for (j = 0 up to clause.size) {
32                    if (j == watches.first or j == watches.second) {
33                        continue
34                    }
35
36                    int unwatched = clause[j]
37                    int unwatched_idx = abs(unwatched)
38
39                    bool evals_false =
40                        (unwatched > 0 and assignment[unwatched_idx]
41                            == False)
42                        or
43                        (unwatched < 0 and assignment[unwatched_idx]
44                            == True)
45                    if (!evals_false) {
46                        append(var_to_clause[unwatched], clause_idx)
47                        erase_at(watched_clauses, i)
```

```
48                          watches.first = j
49                          relocated = True
50                          break
51                      }
52                  }
53
54                  if (relocated == False) {
55                      i = i + 1
56
57                      int other_watch = clause[watches.second];
58                      int idx = abs(other_watch)
59
60                      if (assignment[idx] == Unassigned) {
61                          if (other_watch > 0) {
62                              assignment[idx] = True
63                          } else {
64                              assignment[idx] = False
65                          }
66
67                          append(delta, idx)
68                          enqueue(unitProp, other_watch)
69                      } else {
70                          bool evals_false =
71                              (other_watch > 0 and assignment[idx]
72                                  == False)
73                              or
74                              (other_watch < 0 and assignment[idx]
75                                  == True)
76
77                          if (evals_false == False) {
78                              continue
79                          }
80
81                          for (k = 0 up to delta.size) {
82                              assignment[delta[k]] = Unassigned
83                          }
84
85                          clear(unitProp)
86
87                          return null
88                      }
89                  }
90              }
91          }
92          return delta
93      }
```

This is a huge method, so let's walkthrough it so we understand every aspect of it. If we look back at our *solve()* code, we pass in *cur_var* and *-cur_var* as the *decision*

parameter into the propagation method. And in the beginning of *solveCNF()*, we pass in 0, to signify propagation at the root level. When that happens, we don't add anything into the queue, instead we just propagate on the literals that were forced during the creation of the watched literals.

Like I said before, we only need to check the clauses containing the negative version of the literal being propagated on because those are the clauses that the actual unit propagation could be forced upon. However, we only need to actually check the clauses being watched by the negative version of the literal being propagated on.

Next, we grab the indices of those clauses being watched and then for each one, we try to relocate the index of the first watched literal to a new literal that doesn't evaluate to False. Why only the first? Because swapping the two indices and operating on only the first index is far easier than creating a second branch dedicated to the second index. If a literal doesn't evaluate to False, we first make that unwatched literal now watch the current clause, we then make the current watched literal stop watching the current clause, and then we update the first index of the watched literals to be the index of the newly watching literal. That means we have successfully relocated the watched literal and no unit propagation needs to occur. Because we're modifying the array of clause indices, we don't want to increment the index variable in our implementation.

Aside: When you delete something in a dynamic array, in order to fill the blank space, every element to the right of what you deleted has to shift over to the left by one unit. This takes O(n) time and on large arrays can be very costly. Because the next element has shifted over to the left, its index is now the current index, so you don't want to increment the index variable. In the implementation, the relocation logic is implemented as a "swap and pop" in which we swap the current element with the last one, then we delete the last element, which was the current element. This leads to no shifting of elements, and is logically equivalent (for our use case), even if the order of elements, relative to the deleted element, is no longer the same.

If we couldn't relocate the watched literal's index, we want to increment the index variable so that on the next iteration it will point to a new element. In the case that we couldn't relocate the watched literal, it means that every element in the clause, except for possibly the element pointed to by the second watched literal, evaluates to False. If the element pointed to by the second watched literal also evaluates to False, then we've encountered a contradiction in our assignment. However, if the second watched literal evaluates to True, then the clause is already satisfied and we need to do nothing except for move onto the next variable to propagate on. And if the second watched literal evaluates to Unassigned, then we can assign it the value it needs to evaluate to True and we then propagate on that newly assigned unit literal.

If we encounter a contradiction, we must undo all the propagation we've done, including the initial decision before returning a null value to signify that the propagation failed. However, if it has succeeded, then we return the array of the variable indices that we've assigned.

So one of the big aspects of watched literals is how we get away with doing so little amount of work. And the answer is something called the "watched literal invariant." This invariant is that at all times, at least one of the watched literals doesn't evaluate to False. I don't expect this to make everything click immediately, so let's explain more. A clause is only a contradiction when everything evaluates to False. However,

we don't want to check every single clause when we assign only one variable. And even checking the negative of the variable we assigned is too much. So instead, we change which variables are watching which clauses every now and then. We pick at most two variables to watch a singular clause with, and each one watches their respective clauses until they are assigned a value that makes them evaluate to False. In that case, there is the possibility that the watched literal invariant may be violated (ie, there is a possibility that both watched literals may evaluate to False), but just to be sure, we try to change the variable watching that same clause.

We scan the clause for any unwatched literal that is Unassigned or True, and if we find one, we've restored the watched literal invariant, so we don't need to propagate or do anything else for that matter. However, if we can't find one, then we're in serious danger of the watched literal invariant being violated. So our only hope is for the second watched literal to either already be True (we need to do nothing), or Unassigned (we assign it and propagate).

Thanks to that invariant, we also don't need to do any undoing when we backtrack other than undoing what we propagated. So long as the watched literal invariant holds true when we undo our assignments, the watched literals don't need to move, which means besides the undoing of what we propagated, we get zero cost backtracking.

3.5.3 Why Two Watches?

This now begs the question, why two watched literals? What is special about the number 2? Why can't we have just one watched literal? In order to answer this question, we need to ask ourselves: "what can we do with two watched literals that we can't do with one?" When we have two watched literals, we can efficiently check for whether or not we can propagate. How so?

After we try propagating on a literal, we scan the whole clause except for the indices of the first and second watched literals. Next, if everything in the clause evaluates to False, our only hope to propagate is on the second watched literal. So if that evaluates to False as well, we have found a contradiction, and if it doesn't we have found a literal we can propagate on.

But why is that more efficient that just one watched literal? Let's think of the worst case scenario for if we just had one watched literal. What does that look like? Suppose we were watching the first literal in the clause and every other literal in the clause evaluated to False. Ie, we need to propagate on this watched literal. How would it know it needs to propagate when it was never alerted by anything becoming False? It would have no idea it needs to do anything until that watched literal was falsified, and by that point, everything in the clause would evaluate to False, so the clause would be unsatisfied, only then would we backtrack, but when we backtrack, we'd still have no way of knowing if the clause is unit.

So with only one watched literal, in the worst case scenario, which is that the literal we're watching is the last literal in the clause to be falsified, we'd be doing the same amount of work as normal unit propagation, perhaps even more.

Visually speaking, this is what I'm talking about:

$$UUU$$

The last literal becomes False, but we're not watching it so nothing moves.

$$UUF$$

Next the second literal becomes False, but we're not watching it so nothing happens.

$$UFF$$

Although we should perform unit propagation, our watched literal is completely blind to the fact that it is the last Unassigned literal in the clause, so it remains as is. If luck has it, it could become True and all would be well, but this is reality, so more likely than not, it will also become False.

$$FFF$$

Now we scan the clause, and we see that we can't relocate the watched literal, so we backtrack on our assignment.

$$UFF$$

Since the watched literal was never aware if both of the other literals became False in one pass of the algorithm or if it was across multiple decisions, it still doesn't know that it's the last Unassigned literal. It has to scan the clause each time to know whether or not it's the last Unassigned literal, which is no better than regular unit propagation.

3.5.4 Watched Literals - Results

After implementing watched literals, our runtime decreases dramatically. For uf20.91, solving all 1000 instances takes **349ms**, as opposed to the **1548ms** from pure literal elimination and the **868ms** from normal unit propagation. And for UUF50.218.1000, it takes only **7130ms** as opposed to the **231,107ms** from pure literal elimination and the **140,411ms** from normal unit propagation. Watched literals took us down from minutes to seconds for the unsatisfiable problems, which is amazing for us.

```
On all 1000 equations in uf20-91:

Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

3.6 Iterative DPLL

Now that we've implemented DPLL to the best of our ability, it's time to take it one step further. We will now implement it iteratively rather than recursively. Why? Remember how I said in the beginning that DPLL is the foundation for modern SAT solvers? [17] Well iterative DPLL is the foundation for the next chapter's algorithm: CDCL. Implementing iterative DPLL is imperative in order to create a modern SAT solver worth its salt. [18]

As said in the beginning of the chapter, DPLL forces us to traverse the decision tree as well, a tree. And that is still true, even in iterative DPLL. What changes now is that instead of an implicit path, we now have an explicit one. Suppose we are on the first decision level, so we decide variable $1 = T$, and that forces $3 = F$, $5 = F$, and $6 = T$. In recursive DPLL, we assign these variables into the *assignment* or *vars* field, but we don't really use them until we get to the decision level that matches their index or the evaluation phase.

With iterative DPLL, there are still decision levels, but the representation is now very, very different bcause instead of a decision *tree*, we now use a decision *trail*. The trail is reminiscent of a queue in the sense that we only operate at the end of the data structure. It contains some information that we need in order to continue mimicking the behavior of recursive DPLL. Each trail entry contains the literal that was forced, the decision level it's a part of, and the clause that forced that decision. We append these trail entries every single time we force a variable's assignment. The next question is how do we know which variables were decided for and which variables were inferred from propagation? Well, for decisions, there is no reason clause because there was no reason to begin with other than "because I said so." Therefore, the variables that were decided for don't get a reason clause, but the ones that were inferred do. This is great

[17] I can't remember if I actually said that

[18] Honestly, "Salt" is a better name than "Saturn." I might change it.

because it makes it easy to tell where the beginning of one decision starts and where it ends.

However, it would be nice to have some way to keep track of where the decisions are in the trail other than scanning for entries without a reason clause. Because of that, we will also keep another trail specifically for the decision markers. This is also helpful because then we can store whether or not we've "tried the other branch" when we backtrack.

While on the topic of backtracking, it would be nice to discuss it some more as well. Since we can't simply go up the call stack and undo variable assignments, we have to do it in the iterative way. When we backtrack, we first backtrack up to the decision entry. This is done by popping off the trail and undoing the propagated assignments. Once that's done, you then check if you've "tried the other branch", and if you haven't, you flip the decision literal's assignment and then retry. If you have already tried the other branch, you pop off the decision entry, and then repeat this process to the next decision entry. This process continues until you either find a decision that has not been switched yet, or you run out of decision entries.

With most of the exposition out of the way, I think it would be good to look at the changes we will make to our interface and algorithms, starting with our SAT solver.

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
13        TrailEntry {
14            int lit,
15            int clauseIdx
16        }
17
18        DecisionLevel {
19            int idx,
20            bool triedOtherBranch
21        }
22
23        void parse(string equation)
24        bool succeeded propagate()
25        bool createWatched()
26
27        int numVars
28        int numClauses
29        int trailHead
```

```
30
31          int[][] clauses
32          bool[] satisfied
33
34          TrailEntry[] trail
35          DecisionLevel[] trailStarts
36
37          optional var[] vars
38          map<int, int[]> var_to_clause
39          map<int, (int, int)> clause_to_vars
```

The first notable change is that *propagate()* now returns a boolean instead of a delta. This is because it will no longer be responsible for undoing propagation in response to a contradiction, thus achieving zero-cost backtracking. Next, we now have a *trail* and a *trailHead*, which points to the last made decision entry. *trailStarts* keeps track of all the indices of the decision entries. One last thing to note is that the *TrailEntry* data structure uses indices for clauses instead of the actual clause itself. There's a very good reason for this, and I will reveal the answer in the next chapter, but until then, why do you think it might be that way? Hint: it's not because indices are easy for accessing.

The next change we need to make is to our algorithm to create watched literals. It's the same for basically 90% of the algorithm, except for the unit clause case, which I'll highlight.

```
1       bool createWatched(int[] assignment) {
2           for (i = 0 up to numClauses) {
3               clause = clauses[i]
4               ...
5               else if (clause.size == 1) {
6                   int var = clause[0]
7                   ...
8                   append(trail, {var, i})
9                   ...
10              }
11              ...
12          }
13          return True
14      }
```

Rather than enqueuing the variable, we create a new trail entry for each unit clause. These ones are all part of the **root** decision level, which we mentally reserve for the 0th decision level. If we backtrack up to this level (if it exists) or before it, we've encountered a root level contradiction, so the equation is unsatisfiable. Because not every equation has unit clauses, it's important to remember that this decision level may not always exist, and so we should be keep that in mind for our solving algorithm.

The next algorithm we're going to change is the *propagate()* algorithm because the changes are also very minimal.

```
1       bool propagate(int[] assignment) {
```

```
2           while (trailHead < trail.size) {
3               int prop = trail[trailHead].lit
4               trailHead = trailHead + 1
5               ...
6               for (i = 0 up to watched_clauses.size) {
7                   clause_idx = watched_clauses[i]
8                   int[] clause = clauses[clause_idx]
9                   (int, int) watches = clause_to_var[clause_idx]
10                  ...
11                  if (relocated == False) {
12                      ...
13                      int other_watch = clause[watches.second];
14                      int idx = abs(other_watch)
15                      if (assignment[idx] == Unassigned) {
16                          ...
17                          append(trail, {other_watch, clause_idx})
18                      } else {
19                          return false
20                      }
21                  }
22              }
23          }
24          return true
25      }
```

The main difference is that we just *trailHead* to iterate through the trail, and instead of enqueuing, we just append to the trail. We also return True/False depending on whether or not propagation succeeded.

Lastly, we look at our *solve()* method, which has the most notable changes. There are two ways to implement this, depending on what makes sense to you. I call them "assignment before propagation" and "propagation before assignment." I call them these because that's what they are. Before showing the pseudocode, I want to talk about the algorithm a bit more. The beginning is mostly the same: create our watched literals, check that it succeeded, return False if it didn't. Otherwise, we do a root level propagation and then check if it succeeds as well. However, the main confusion comes from handling the case of when there is a 0th decision level and when there isn't. There are multiple ways to go about this. After the beginning, we enter the solving loop.

There are two main parts: assigning and backtracking, with a call to the propagate method separating them. Assigning is now scanning for the first Unassigned variable, forcing it True, creating a new trail entry, and then updating the trail head. I'm sure there's a better way to keep track of the next unassigned variable other than scanning the entire assignment list on each loop iteration, however, for simplicity's sake, we're implementing it the naive way.

Since we've already discussed backtracking, we will now get into the algorithm itself:

```
1       bool solve() {
2           var[1..numVars] assignment
```

```
3
4           for each (var in assignment) {
5               var = Unassigned
6           }
7
8           if (createWatched(assignment) == False) {
9               return False
10          }
11
12          if (trail.size > 0) {
13              append(trailStarts, {0, true})
14
15              if (propagate(assignment, 0) == False) {
16                  return False
17              }
18          }
19
20          while (True) {
21              bool succeeded = propagate(assignment)
22
23              if (!succeeded) {
24                  while (trailStarts.size > 0) {
25                      DecisionLevel decision = trailStarts.last
26
27                      while (trail.size > decision.idx + 1) {
28                          int lit = trail.last.lit
29                          assignment[abs(lit)] = Unassigned
30                          erase(trail, trail.last)
31                      }
32
33                      if (decision.triedFalse == False) {
34                          TrailEntry entry = trail.last
35
36                          entry.lit = -entry.lit
37                          assignment[abs(entry.lit)] = False
38                          decision.triedFalse = True
39
40                          trailHead = decision.idx
41                          break
42                      } else {
43                          int lit = trail.last.lit
44                          assignment[abs(lit)] = Unassigned
45                          erase(trail.last)
46                          erase(trailStarts.last)
47                      }
48                  }
49
50                  if (trailStarts.size == 0) {
51                      return False
52                  }
```

```
53
54                  continue
55              }
56          }
57
58          //Assignment code
59          int firstUnassigned = -1
60          for (i = 1 up to numVars) {
61              if (assignment[i] == Unassigned) {
62                  firstUnassigned = i
63                  break
64              }
65          }
66
67          if (firstUnassigned == -1) {
68              return True
69          }
70
71          assignment[firstUnassigned] = True
72          append(trail, {firstUnassigned, -1})
73          append(trailStarts, {trail.size - 1, False})
74          trailHead = trailStarts.last.idx
75      }
```

Here is the pseudocode for solving, written as “propagate then assign." Quick question: when there’s no 0th decision level and we propagate, how do we know that none of the propagated variables will have the wrong decision level? Ie, how do we know that they’ll have a decision level of 0 instead of 1, since we haven’t made any decisions yet? This is kind of a trick question, the answer is that if there is no 0th level, *propagate()* will do nothing because the while-loop condition will immediately return False, and so the method will terminate, assigning nothing!

Instead of showing you the code for “assign then propagate” (it is NOT just taking the assigning code and moving it up before the call to *propagate()*), let’s run through how this works. When there’s a 0th level, we obviously want to mark it in our *trailStart*, but why do we want to mark that the “other branch” has already been tried? Well that’s because those assigned by the root level propagation are forced into those values, so it doesn’t make sense to swap their values to be their opposites since it will immediately be an unsatisfiable assignment anyways.

Next, in the main solving loop, the backtracking code is just “grab the last decision index, undo until the root decision, if it’s already tried False, undo the decision entirely and repeat with the previous one, and if not, try False and continue.”

Lastly, let’s talk about doing assign then propagation. Here’s why it doesn’t make total sense to start out with propagation:

- we do a root level propagation if there needs to be one.
- if there is no root level propagation to be done, it still doesn’t make sense to immediately propagate since there’s nothing to propagate.

- if there was a root level propagation, it doesn't make sense to do another one immediately since we've just completed one.

So if you wanted to do assign and then propagate, how would that be done? Earlier, I said that it's not just taking the code for assigning and then moving it before propagation, but why? Well, suppose we fail a propagation. We backtrack, and then suppose we haven't tried the other branch for that decision. We encounter a *continue* statement and we go back to the top. If we simply move the assigning code up to the top of the loop, what's the issue? Well remember, we just flipped a decision, so we should actually be ***propagating*** instead of assigning. If we assign immediately after flipping the decision, we now have two decisions at the same time, and that's bad because what if they are contradictory to one another, ie one would force a variable False and the other True? Unfortunately, the propagation code wouldn't catch that because it assumes everything in the trail is valid, but we just made the trail invalid. So to the propagation code, instead of it catching that a variable is forced to be False, but it's True, or vice versa, it may just ignore it because it focuses mainly on Unassigned and False literals.

So if we want to do assign then propagate, how do we make sure that we don't accidentally push something onto the trail when we shouldn't have? We need to wrap the assigning code in a conditional statement that checks that *trailHead == trail.size*, if it doesn't, then that means that there's something to be propagated on because we're not at the end of the trail, but if it does, then we've reached the end, so we should check if we've assigned everything and can be done, or if we need to assign something new.

3.6.1 Iterative DPLL - Results

Here are the results after this huge refactor:
(Assignment before propagation)
uf20.91: **328ms**, down 21ms from **349ms** by Watched Literals
UUF50.218.1000: **6369ms**, down 761ms from **7130ms** by Watched Literals
(Propagation before assignment)
uf20.91: **324ms**
UUF50.218.100: **6365ms**

```
On all 1000 equations in uf20-91:

Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

Should you do propagation before assignment simply because it's faster? It really doesn't matter, you do you.

The speed gain on satisfiable problems is very miniscule, however, if you ever want proof that iterative algorithms are faster than recursive ones, the unsatisfiable problem time decrease is pretty good evidence.

Chapter 4

CDCL

Conflict Driven Clause Learning (CDCL), is *the* modern SAT solving algorithm. It builds off of the ideas of DPLL and takes it to a whole new level. The "conflict driven" part of CDCL means that we analyze conflicts (contradictions) and see what we can learn from them. The "clause learning" part is exactly what it sounds like, we add new clauses to our database. In other words, CDCL is about analyzing why conflicts occur when we solve and using those conflicts to learn new clauses that will help enforce new rules as to what values variables can take, essentially changing the CNF equation into one that's logically equivalent, but easier to solve. I won't lie to you, I was not able to implement this algorithm the first time I tried. I hope and pray that this time I will be successful, so that *we* will be successful together dear reader.

Aside: why have we been opting for indices so often? You see, although we live in the land of pseudocode or machine-agnosticism, implementations don't (duh). So in our previous chapter in DPLL, there were times where we used clause indices instead of pointers to clauses (obviously using copies would be terrible for performance and memory), but why did we use indices? Now that we're implementing CDCL, we're going to be adding clauses to the database quite often, and that can trigger a resize on the database. If we used pointers to clauses, after a resize triggers, the memory address of each clause would change, thus making the pointers invalid. We would need to add new code after the insertion of each clause to make sure a resize didn't trigger, and if one did, we'd need to update each pointer for each reason clause and watched literal, which would be very difficult because after a resize, we now have no way to tell which clauses mapped to which reasons and watched literals, because, well, the old mapping isn't valid and contains no information about the new addresses for everything.
[1]

[1]More information about CDCL can be found here: `https://www.cis.upenn.edu/~cis1890/files/Lecture4.pdf`

4.1 Clause Learning

The first step into creating a CDCL solver is to learn and use new clauses. This is easier said than done. The first thing we need to do is modify our unit propagation code to return a clause when there is a conflict and nothing otherwise. What clause do we return when there is a conflict? We return the clause that evaluates to False because that's where we have a contradiction in our assignment. And just as a reminder, the main solving loop now handles undoing the assignments of the literals in our trail, so we don't need to do anything other than return the clause that's evaluating to False.

The next step is to perform something called "clause resolution" over and over until a certain stopping point is reached. Since we're not implementing backjumping nor first UIPs yet, that stopping point is going to be when our learned clause contains only decision literals. Here is how clause resolution works:

Assuming we've set up storing reason clauses and contradiction clauses correctly, We create a copy of the contradiction clause, find the most recently propagated literal within it, take its reason clause, union it with the contradiction clause, remove the most recently propagated literal and its negation, remove all duplicate literals, and repeat until there are no more propagated literals in the contradiction clause. Afterwards, we backtrack one level, add this clause to our database, and then initialize its watched literals.

So let's now understand the algorithmic portions. Because we want to be able to find the most recently propagated literal in a clause at all times, it would be nice to have a data structure that is a mapping from variables to their places in the decision trail.

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
13        TrailEntry {
14            int lit,
15            int clauseIdx
16        }
17
18        DecisionLevel {
19            int idx,
20            bool triedOtherBranch
21        }
22
23        void parse(string equation)
24        bool succeeded propagate()
```

```
25          bool createWatched()
26          bool createWatched(int idx)
27
28          int numVars
29          int numClauses
30          int trailHead
31
32          int[][] clauses
33
34          TrailEntry[] trail
35          DecisionLevel[] trailStarts
36          int[] var_to_trail
37
38          optional<var>[] vars
39          map<int, int[]> var_to_clause
40          map<int, (int, int)> clause_to_vars
```

The *var_to_trail* field is the only change we'll need to make, and it is just a mapping from variable index to trail index. If you enforce the invariant that each variable maps to exactly one place in the trail and no two variables map to the same place in the trail at the same time, you don't need to initialize these values to anything.

The next thing we'll look at is our *propagate()* code since it needs very minimal changes. The change we're making now is that instead of returning a boolean value to indicate whether or not propagation was successful, we're now going to return a clause if we encounter a contradiction and nothing otherwise.

```
1       bool propagate(int[] assignment) {
2           while (trailHead < trail.size) {
3               int prop = trail[trailHead].lit
4               trailHead = trailHead + 1
5               ...
6               for (i = 0 up to watched_clauses.size) {
7                   clause_idx = watched_clauses[i]
8                   int[] clause = clauses[clause_idx]
9                   (int, int) watches = clause_to_var[clause_idx]
10                  ...
11                  if (relocated == False) {
12                      ...
13                      int other_watch = clause[watches.second];
14                      int idx = abs(other_watch)
15                      if (assignment[idx] == Unassigned) {
16                          ...
17                          append(trail, {other_watch, clause_idx})
18                      } else {
19                          return clause
20                      }
21                  }
22              }
23          }
```

```
24          return null
25      }
```

And before we look at changes to our solving method, we're going to create a new overload for *createWatched()* that takes in the index of a clause. Its only purpose is to be called on newly learned clauses, which don't have the same guarantees as the other overload. You see, the learned clause should hopefully NEVER be unsatisfied when this method operates over it. It should ALWAYS return True, so if it ever doesn't, it's a good debugging tool. Here's how it works: we traverse over the clause of the passed in index, counting the number of True, False, and Unassigned literals it has. We will then search for two indices that could be watched literals. Since the clause should never be unsatisfied, we are guaranteed to always find one (if our implementation is correct). If we can't find another and our clause has more than one literal, we will initialize the second watched literal to any arbitrary place in the clause (with the caveat that it can't be the same index as our first watched literal). We will then check if the clause is unit, and if it is, we will add it to our trail, assign the Unassigned variable, etc. Otherwise, we do nothing.

```
1       bool createWatched(int index) {
2           clause = clauses[index]
3           if (clause.size == 0) {
4               return False
5           }
6
7           bool foundFirst = False
8           bool foundSecond = False
9           int firstIdx = -1
10          int secondIdx = -1
11
12          int numUnassigned = 0
13          int numFalse = 0
14          int numTrue = 0
15
16          for (i = 1 up to clause.size) {
17              int idx = abs(clause[i])
18              if (assignment[idx] == Unassigned) {
19                  numUnassigned = numUnassigned + 1
20                  if (foundFirst == False) {
21                      foundFirst = True
22                      firstIdx = i
23                  } else if (foundSecond == False) {
24                      foundSecond = True
25                      secondIdx = i
26                  }
27              } else if (evals_false(clause[i])) {
28                  numFalse = numFalse + 1
29              } else {
30                  numTrue = numTrue + 1
```

```
31
32                 if (foundFirst == False) {
33                     foundFirst = True
34                     firstIdx = i
35                 } else if (foundSecond == False) {
36                     foundSecond = True
37                     secondIdx = i
38                 }
39             }
40         }
41
42         if (clause.size >= 2) {
43             if (foundSecond == False) {
44                 secondIdx = (firstIdx + 1) % clause.size
45             }
46
47             append(var_to_clause[clause[firstIdx]], index)
48             append(var_to_clause[clause[secondIdx]], index)
49
50             clause_to_var[index] = {firstIdx, secondIdx}
51         } else {
52             append(var_to_clause[clause[firstIdx]])
53             clause_to_var[index] = {firstIdx, firstIdx}
54         }
55
56         if (numTrue == 0 && numUnassigned == 1) {
57             append(trail, {clause[firstIdx], index})
58             var_to_trail[abs(clause[firstIdx])] = trail.size() - 1
59
60             int var = clause[firstIdx]
61             int idx = abs(var)
62
63             if (var > 0) {
64                 assignment[idx] = True
65             } else {
66                 assignment[idx] = False
67             }
68         }
69
70         return True
71     }
```

Because we don't know anything about this clause, we need to gather as much information as we can right away before we decide how watched literals are spread out on the clause. When the clause has more than one literal, we use "secondIdx = (firstIdx + 1) % clause.size" because that prevents us from adding a watched literal past the end of the clause, if *firstIdx* is already on the edge.

The last method we need to discuss is our *solve* loop. Clause resolution happens

after backtracking because it relies on the current trail to understand [2] why a conflict occurred. If we were to undo propagations, then the resolution operator would have no reference as to what "most recently" means, and therefore, clause resolution wouldn't work.

```
1     bool solve() {
2         var[1..numVars] assignment
3
4         for each (var in assignment) {
5             var = Unassigned
6         }
7
8         if (createWatched(assignment) == False) {
9             return False
10        }
11
12        if (trail.size > 0) {
13            append(trailStarts, {0, true})
14
15            if (propagate(assignment, 0) == False) {
16                return False
17            }
18        }
19
20        while (True) {
21            optional int[] contradiction = propagate(assignment)
22
23            if (contradiction has a value) {
24                int[] conflict = contradiction.value
25
26                bool propagated_found = True
27                while (True) {
28                    propagated_found = False
29                    int idx = 0
30
31                    for (i = 1 up to conflict.size) {
32                        int var = conflict[i]
33                        int v_idx = abs(var)
34
35                        int t_idx = var_to_trail[v_idx]
36                        if (trail[t_idx].reasonIdx has a value) {
37                            propagated_found = True
38                            idx = max(idx, t_idx)
39                        }
40                    }
41
```

[2] It follows a mathematical process to transform the conflict clause into one that prevents that same conflict from ever occurring again. The algorithm does not "understand" anything. Do not anthropomorphize computer algorithms, especially artificial intelligence.

```
42                      if (propagated_found == False) {
43                          break
44                      }
45
46                      int lit = trail[idx].lit
47                      int[] reason = clauses[trail[idx].reasonIdx]
48
49                      append(conflict, reason)
50
51                      for (i = 0 up to conflict.size) {
52                          if (abs(conflict[i]) == abs(lit)) {
53                              swap(conflict[i], conflict.last)
54                              erase(conflict, conflict.last)
55                          } else {
56                              i = i + 1
57                          }
58                      }
59
60                      set<int> duplicates
61                      for (i = 0 up to conflict.size) {
62                          if (contains(duplicates, conflict[i])) {
63                              swap(conflict[i], conflict.last)
64                              erase(conflict, conflict.last)
65                          } else {
66                              insert(duplicates, conflict[i])
67                              i = i + 1
68                          }
69                      }
70                  }
71
72                  while (trailStarts.size > 0) {
73                      DecisionLevel decision = trailStarts.last
74
75                      while (trail.size > decision.idx + 1) {
76                          int lit = trail.last.lit
77                          assignment[abs(lit)] = Unassigned
78                          var_to_trail[abs(lit)] = -1
79                          erase(trail, trail.last)
80                      }
81
82                      if (decision.triedFalse == False) {
83                          TrailEntry entry = trail.last
84
85                          entry.lit = -entry.lit
86                          assignment[abs(entry.lit)] = False
87                          var_to_trail[abs(entry.lit)] = -1
88                          decision.triedFalse = True
89
90                          trailHead = decision.idx
91                          break
```

```
92                      } else {
93                          int lit = trail.last.lit
94                          assignment[abs(lit)] = Unassigned
95                          var_to_trail[abs(lit)] = -1
96                          erase(trail.last)
97                          erase(trailStarts.last)
98                      }
99                  }
100
101                 if (trailStarts.size == 0) {
102                     return False
103                 }
104
105                 append(clauses, conflict)
106                 createWatched(clauses.size)
107
108                 continue
109             }
110         }
111
112         //Assignment code
113         int firstUnassigned = -1
114         for (i = 1 up to numVars) {
115             if (assignment[i] == Unassigned) {
116                 firstUnassigned = i
117                 break
118             }
119         }
120
121         if (firstUnassigned == -1) {
122             return True
123         }
124
125         assignment[firstUnassigned] = True
126         append(trail, {firstUnassigned, -1})
127         append(trailStarts, {trail.size - 1, False})
128         trailHead = trailStarts.last.idx
129     }
```

And with that, we've finished clause learning.

4.1.1 Clause Learning - Results

Here are the results for Clause Learning:

uf20.91: 425ms, up 98ms from Iterative DPLL

UUF50.218.1000: 9652ms, up 3287ms from Iterative DPLL

Why did our runtime go up? Well, clause learning isn't easy, and the more contradictions we encounter, the more expensive clause learning we're performing. Another reason is that we're increasing the memory load every time we add in a new clause.

Luckily, our testing sets are small, but on large problems, this would be a much larger issue. So then why do modern solvers use clause learning? Because there are ways to make it much faster. The first step would be to implement non-chronological backjumping, where instead of moving up just one decision level in our trail, we move up multiple at a time. The second step would be to implement First UIPs, which makes clause learning a less expensive operation (doesn't fix the memory load though), and it combined with non chronological backjumping allows us to instantly and fully use our learned clause to its potential. Although we have taken a small step back, we will see what leaps forward we will make in the next sections.

4.2 Non-Chronological Backjumping

4.2.1 Non-Chronological Backjumping - Results

Okay so I know I said that we would make leaps forward in the next sections. I may have lied. Here are the results for Non-Chronological Backjumping"

uf20.91: 380ms, down 45ms from Clause Learning.

UUF50.218.1000: 11291ms, up 1639ms from Clause Learning.

So why did our time to solve unsatisfiable equations go up? It partially has to do with the added number of clauses to memory, however, it also has to do with the fact that I may have bunked up the implementation of Clause Learning to begin with. But we're not going to talk about that. You see, when you learn a clause, how can you tell if the equation will be unsatisfiable? If the clause is empty of course, and that means going all the way back to the root decision level. Essentially, even though we're implementing a (supposedly) better algorithm, the amount of backjumping we have to do anyways isn't making huge gains, and in fact can hurt. We will see how First UIPs can help us (hopefully). [3]

4.3 First UIP

4.4 VSIDS

4.5 Clause Database Management

4.6 Restarts

4.7 Comparing to MiniSAT

[3] This project is killing me why did I decide to do it?

Chapter 5

Experiments and Next Steps